Reporte Técnico de Actividades Práctico-Experimentales Nro. 006

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Maria Chuico Carlos Granda Fabricio Ruiz Juan Veintimilla
Asignatura	Sistemas Distribuidos
Ciclo	6 to
Unidad	1
Resultado de aprendizaje de la unidad	R2. Escribe un programa que realiza cualquier requerimiento de clasificación y conversión en unidades de mensajes, como paquetes, para comunicar información interesante entre dos hosts, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad
Práctica Nro.	006
Título de la Práctica	Mensajería Distribuida con RabbitMQ y MQTT. PARTE 2: MICROSERVICIOS CON RABBITMQ Y SPRING BOOT
Nombre del Docente	José O. Guamán Q.
Fecha	27 de octubre de 2026 30 de octubre de 2026
Horario	Martes 10h30 a 11h30 Viernes 07h30 a 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica

Implementar microservicios en Spring Boot con Spring AMQP que consuman las colas de RabbitMQ generadas en la Parte 1, configuren exchanges y bindings, envíen alertas a una cola de notificaciones y expongan una API REST para consultar el estado de la flota.

- Comprender los fundamentos del protocolo MQTT y su aplicación en entornos IoT
- Configurar e instalar un broker Mosquitto para mensajería
- publicador/suscriptor
- Implementar publicadores y suscriptores MQTT en Python usando paho-mqtt
- Diseñar topic hierarchies para organizar la telemetría de una flota vehicular

- Entender el protocolo AMQP 0-9-1 y el modelo de intercambio de RabbitMQ
- Configurar exchanges, colas y bindings en RabbitMQ
- Implementar microservicios en Spring Boot que consuman mensajes de RabbitMQ
- Integrar MQTT y RabbitMQ mediante un servicio puente en Python
- Aplicar patrones de mensajería: pub/sub, routing y competing consumers.

3. Materiales y Reactivos

Guia practica

Software

- Python
- paho-mqtt
- pika
- Java JDK 17
- Spring Boot
- Docker
- Docker Compose
- Mosquito
- RabbitMQ
- SQLite3
- IDE
- Postman
- Navegador web

Hardware

- Computador

4. Procedimiento / Metodología Ejecutada

1) Para empezar la práctica, primero preparamos el entorno y revisamos que tuviéramos todo lo necesario para armar el sistema distribuido: Docker, RabbitMQ, Mosquitto, Python y Spring Boot. Levantamos RabbitMQ y Mosquitto en contenedores Docker y comprobamos que arrancaran bien y se mantuvieran activos.
2) Con el entorno listo, pasamos a configurar RabbitMQ. Ahí creamos el exchange exchange.fleet de tipo direct y armamos los bindings para que los mensajes lleguen a sus colas, usando las routing keys gps.routing, temp.alert y fuel.routing. Así quedaron separados los datos de ubicación, temperatura y combustible de los vehículos.
3) Después ejecutamos la aplicación en Spring Boot y la conectamos a RabbitMQ con Spring AMQP. Verificamos que se conectara bien al broker y que los componentes que reciben y procesan los mensajes estuvieran listos.
4) Luego pusimos en funcionamiento el simulador de sensores, que genera datos de los vehículos VH-001, VH-002 y VH-003: coordenadas GPS, velocidad, temperatura y nivel de combustible. Toda esa información se publica por MQTT hacia Mosquitto.
5) Enseguida ejecutamos el servicio puente en Python, que recibe los mensajes de MQTT y los reenvía a RabbitMQ. En ese paso también se clasifican los datos para mandarlos a la cola que corresponde, y así se comunican los dos sistemas de mensajería.

6) Con todo conectado, probamos los consumidores de Spring Boot. Comprobamos que GpsConsumer recibiera los mensajes de ubicación y que AlertConsumer procesara los de temperatura y combustible. También confirmamos que se generarán alertas cuando la temperatura pase de 4 °C o cuando el combustible baje del 20 %.
7) Después entramos a la interfaz de administración de RabbitMQ para ver que se hubieran creado las cuatro colas del sistema: cola.gps.telemetria, cola.alertas.temperatura, cola.combustible.nivel y cola.notificaciones. En la de notificaciones había 18 mensajes pendientes, lo que confirmó que las alertas se estaban publicando bien.
8) Por último, probamos la API REST con los endpoints de FleetController. Consultamos el estado general de la flota con /api/fleet/status y la telemetría del vehículo VH-001 con /api/fleet/vehicle/VH-001/telemetria. Con eso comprobamos que la aplicación también puede dar información del sistema por HTTP, además de la comunicación por RabbitMQ.
9) Durante todo el proceso fuimos revisando los registros de ejecución, el estado de los contenedores, la llegada de los mensajes y las respuestas de los endpoints, para asegurarnos de que cada parte funcionara como debía.

5. Resultados

Durante la práctica se realizó un sistema distribuido para monitorear una flota vehicular, integrando MQTT, Mosquitto, RabbitMQ y microservicios desarrollados con Spring Boot. Mediante las pruebas realizadas se verificó la comunicación entre los componentes, la recepción de datos de los sensores, el procesamiento de alertas y la consulta de información mediante una API REST.

- **Implementación y conexión de los servicios.**

  Se implementaron los servicios de RabbitMQ y Mosquitto a través de Docker, viendo que ambos contenedores permanecieran activos y disponibles para la comunicación. Se ejecutó la aplicación Spring Boot, comprobando que estableciera correctamente la conexión con RabbitMQ y quedará disponible para recibir y procesar los mensajes.

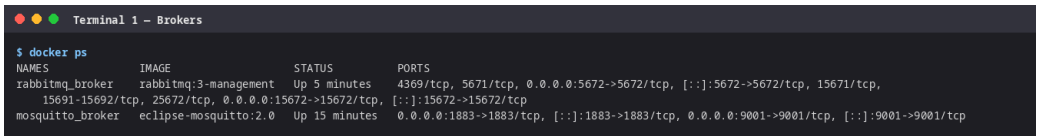


Figura 1. Contenedores RabbitMQ y Mosquitto activos.

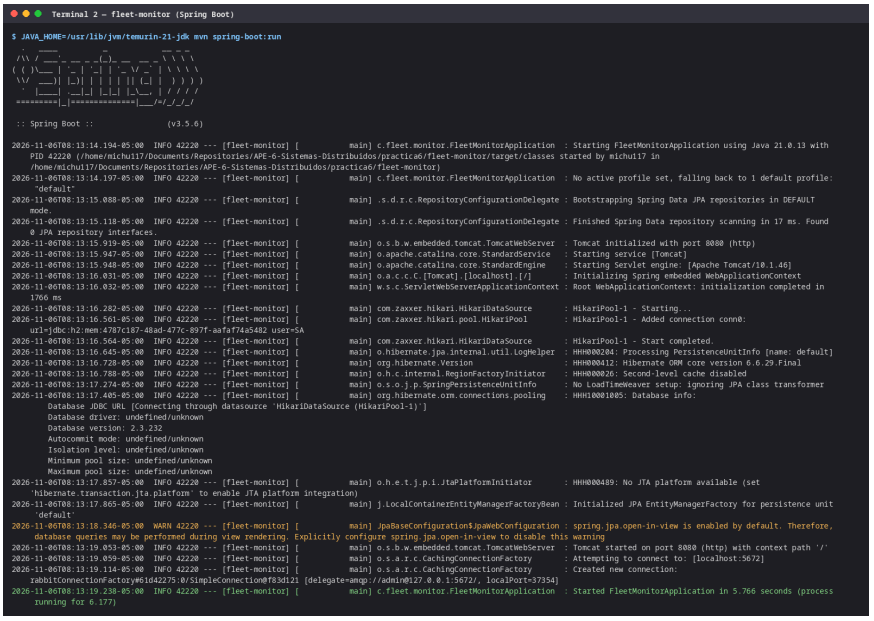


Figura 2. Aplicación Spring Boot conectada a RabbitMQ.

- **Configuración del enrutamiento de mensajes**

  Se configuró el exchange exchange.fleet de tipo direct en RabbitMQ, estableciendo los bindings para distribuir los mensajes según su routing key. Se definieron las siguientes asociaciones: gps.routing para la cola cola.gps.telemetria, temp.alert para cola.alertas.temperatura y fuel.routing para cola.combustible.nivel.

  Esta configuración permite organizar y dirigir los mensajes hacia las colas correspondientes.

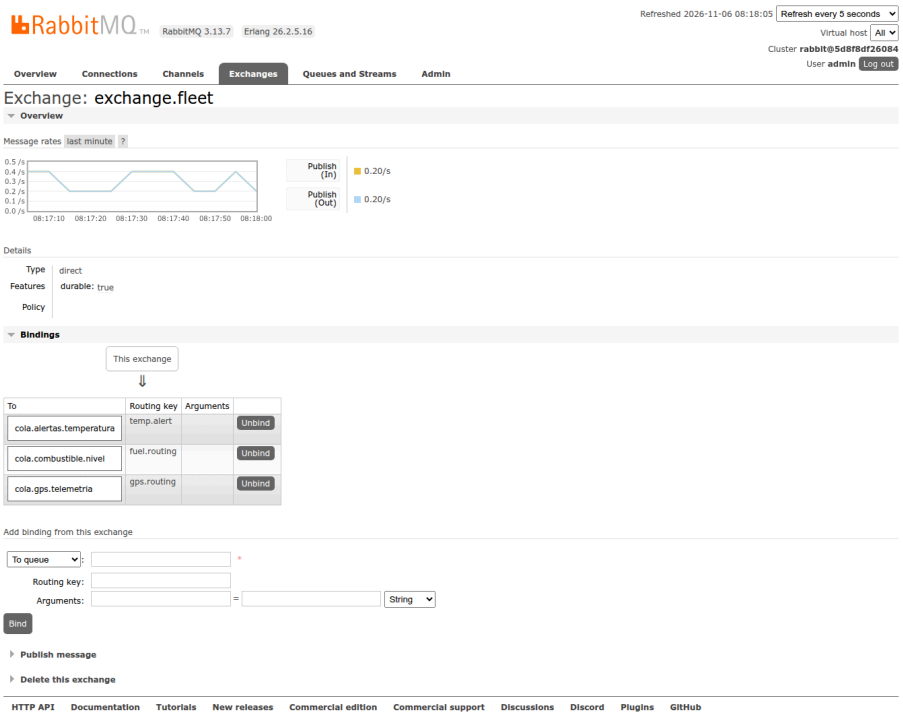


Figura 3. Configuración de bindings del exchange exchange.fleet.

- **Simulación y transmisión de datos mediante MQTT.**

  Se ejecutó el simulador de sensores encargado de generar información de los vehículos VH-001, VH-002 y VH-003. Los datos transmitidos incluyeron coordenadas GPS, velocidad, temperatura y nivel de combustible.

  Los mensajes fueron publicados mediante MQTT hacia el broker Mosquitto. El puente desarrollado en Python recibió la información y la reenvió a RabbitMQ, realizando la clasificación para su distribución.

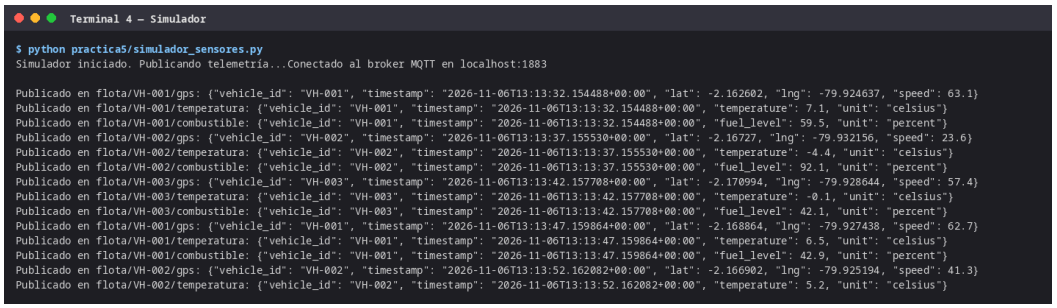


Figura 4. Ejecución del simulador de sensores vehiculares.

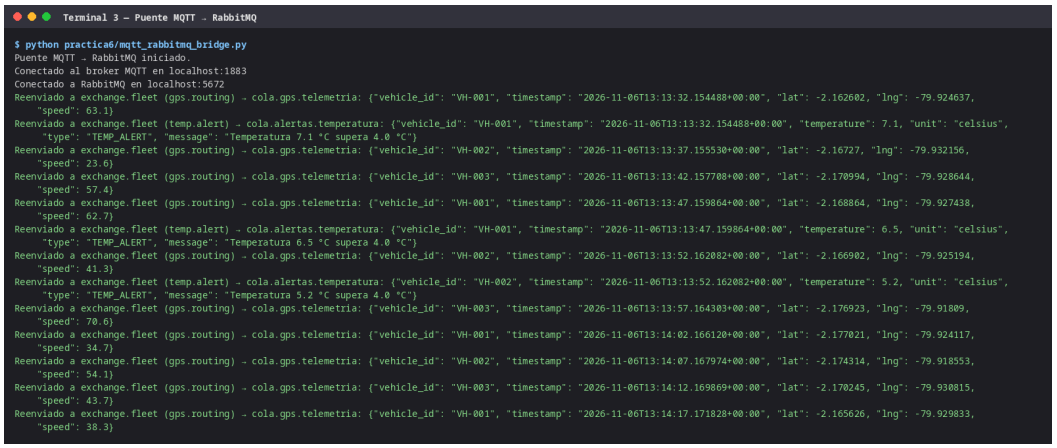


Figura 5. Funcionamiento del puente de comunicación MQTT-RabbitMQ.

- **Procesamiento de telemetría y generación de alertas.**

  Se verificó el funcionamiento de los consumidores implementados en Spring Boot. El componente GpsConsumer recibió los mensajes correspondientes a la ubicación de los vehículos y mostró la información de telemetría en los registros de ejecución.

  El componente AlertConsumer procesó las condiciones relacionadas con la temperatura y el combustible. Durante las pruebas se identificaron alertas cuando la temperatura superó los 4 °C y cuando el nivel de combustible desciende por debajo del 20 %.

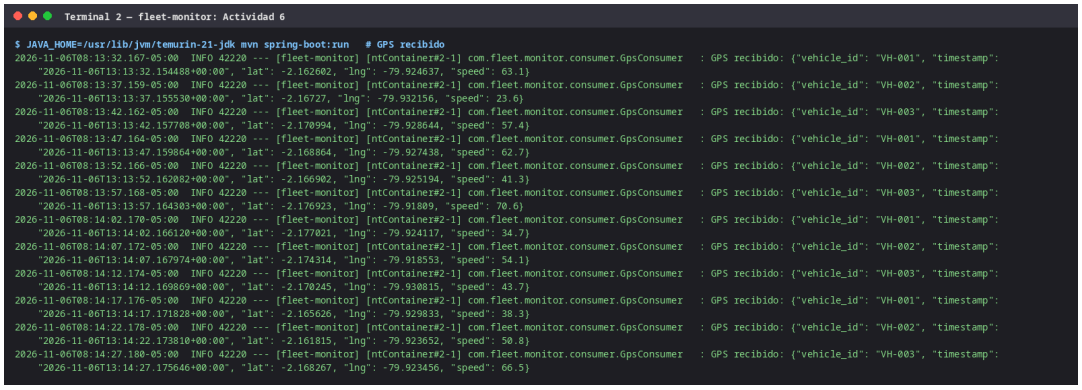


Figura 6. Recepción y procesamiento de datos mediante GpsConsumer.

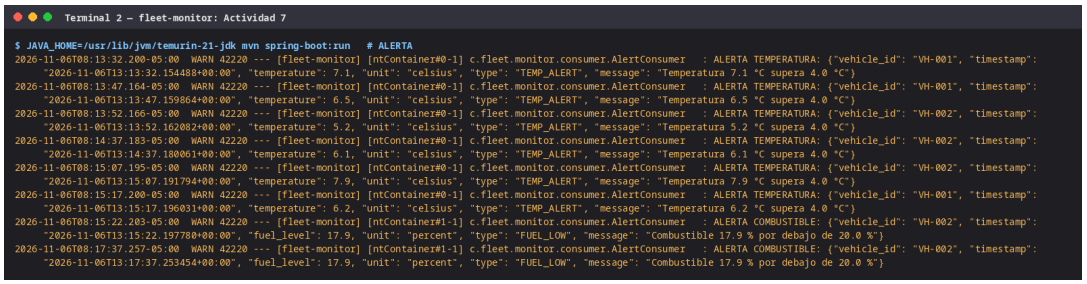


Figura 7. Generación de alertas de temperatura y combustible mediante AlertConsumer.

- **Estructura del proyecto y configuraciónVerificación de colas y mensajes de notificación.**

  Mediante la interfaz de administración de RabbitMQ se verificó la creación de las cuatro colas utilizadas en el sistema: cola.gps.telemetria, cola.alertas.temperatura, cola.combustible.nivel y cola.notificaciones.

  Durante las pruebas se observaron 18 mensajes pendientes en la cola de notificaciones,esto como evidencia que las alertas habían sido publicadas y permanecían almacenadas para luego su procesamiento por el consumidor de notificaciones.

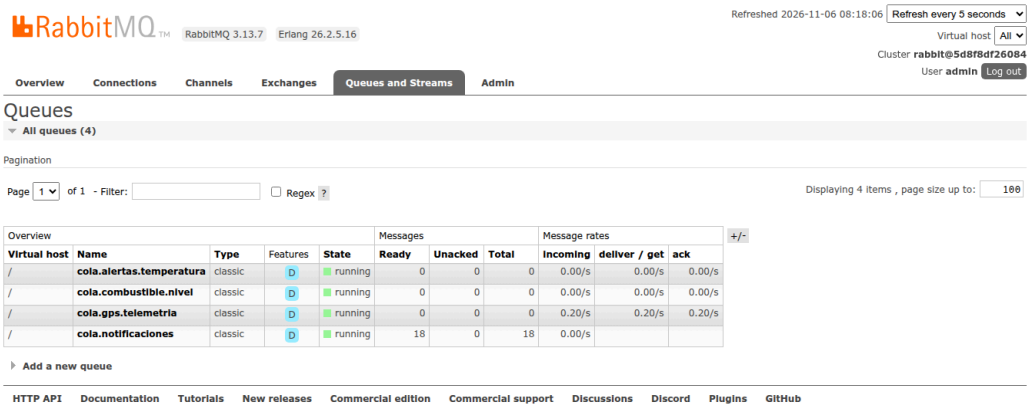


Figura 8. Colas configuradas en RabbitMQ.

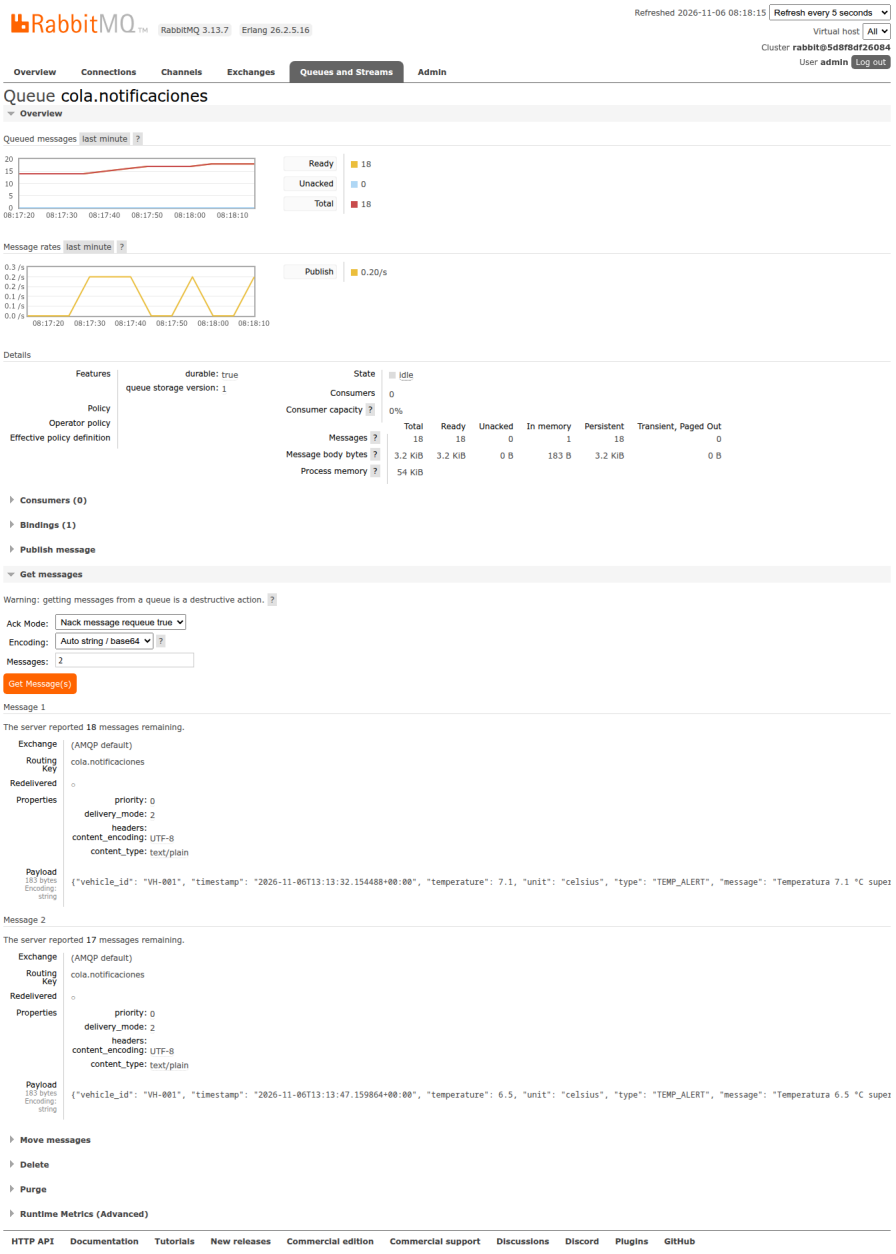


Figura 9. Mensajes pendientes en la cola cola.notificaciones.

- **Pruebas de la API REST.**

  Se comprobó el funcionamiento de la API REST implementada en FleetController, mediante consultas a los endpoints disponibles. Se realizaron pruebas para consultar el estado general de la flota mediante /api/fleet/status y obtener la información de telemetría de un vehículo específico mediante /api/fleet/vehicle/VH-001/telemetria.

  Estas pruebas dejaron verificar que la aplicación exponía información del sistema mediante solicitudes HTTP, complementando el procesamiento de mensajes realizado a través de RabbitMQ.

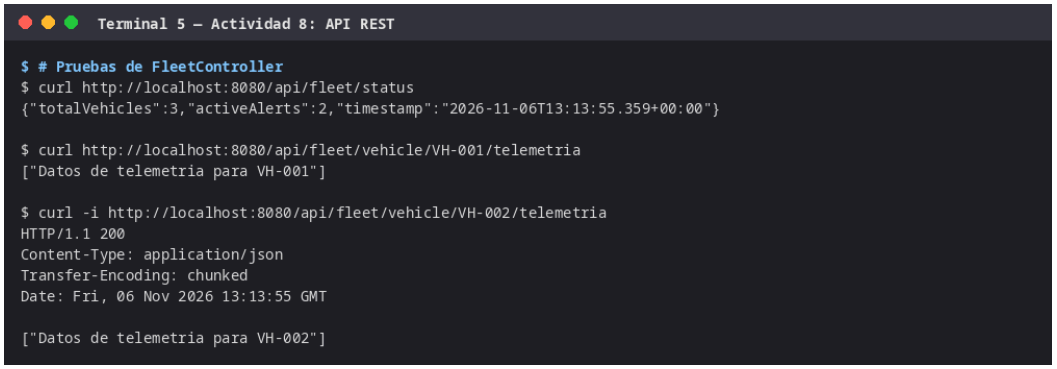


Figura 10. Pruebas de los endpoints de la API REST implementada en FleetController.

Despliegue distribuido en cuatro computadoras

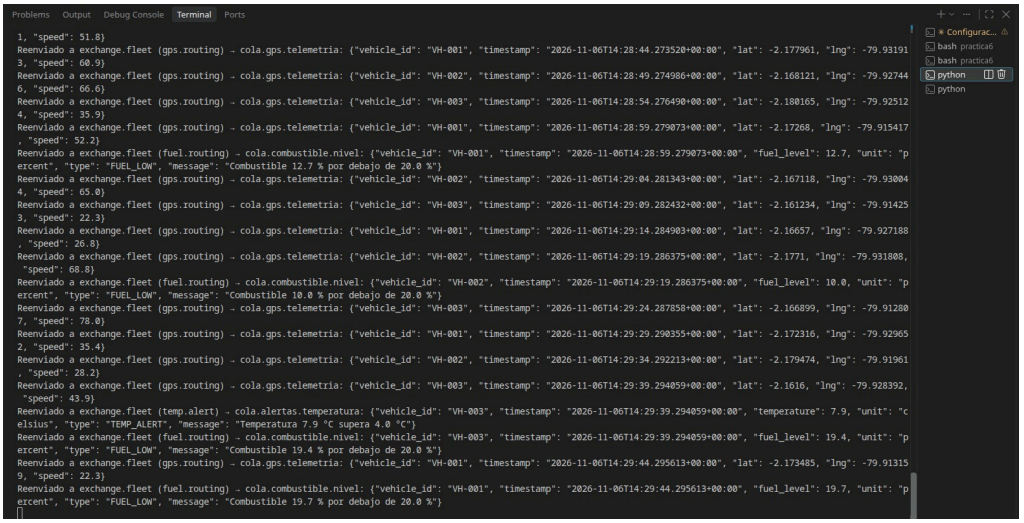


Figura 11. PC 1 (Fabricio Ruiz): simulador de sensores publicando telemetría por MQTT.

En la PC 1 se ejecutó el simulador de los sensores que generó datos de los vehiculos (VH-001 - VH-002 - VH-003) cada 5s. En la imagen se muestran los mensajes publicados en los topics: *flota/VH-00X/gps*, *flota/VH-00X/temperatura y flota/VH-00X/combustible* con su marca de tiempo, el valor medido y unidad.

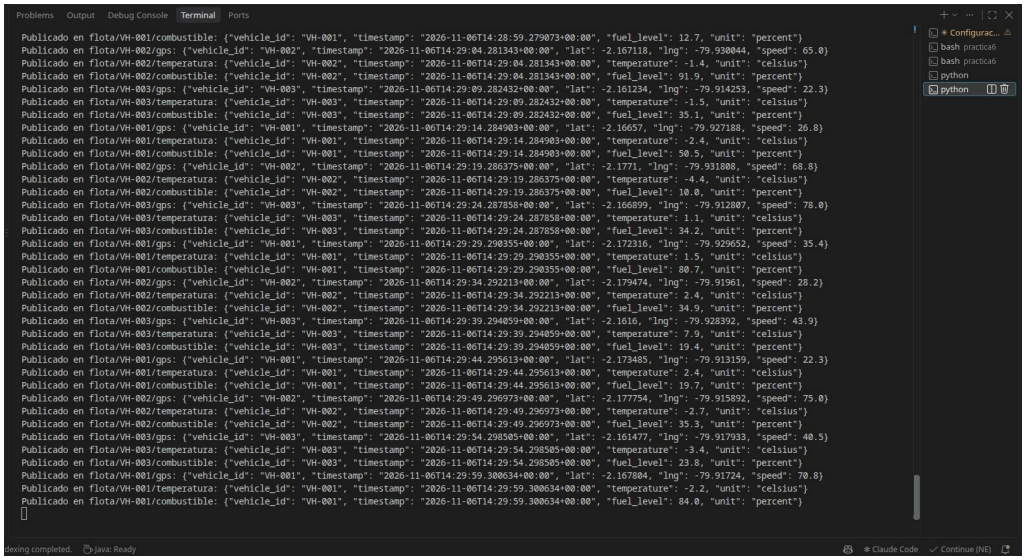


Figura 12. PC 1 (Fabricio Ruiz): puente reenviando los mensajes de MQTT a RabbitMQ.

Así también en la PC 1 se ejecutó el puente en Python que recibió los mensajes de Mosquitto y los reenvió a *exchange.fleet*. En la captura se muestra como los datos se clasifican por routing key: los de ubicación van a *cola.gps.telemetria* con *gps.routing*. Las temperaturas superiores a 4 °C van a *cola.alertas.temperatura* con *temp.alert* y los niveles de combustible inferiores al 20 % van a *cola.combustible.nivel* con *fuel.routing*.

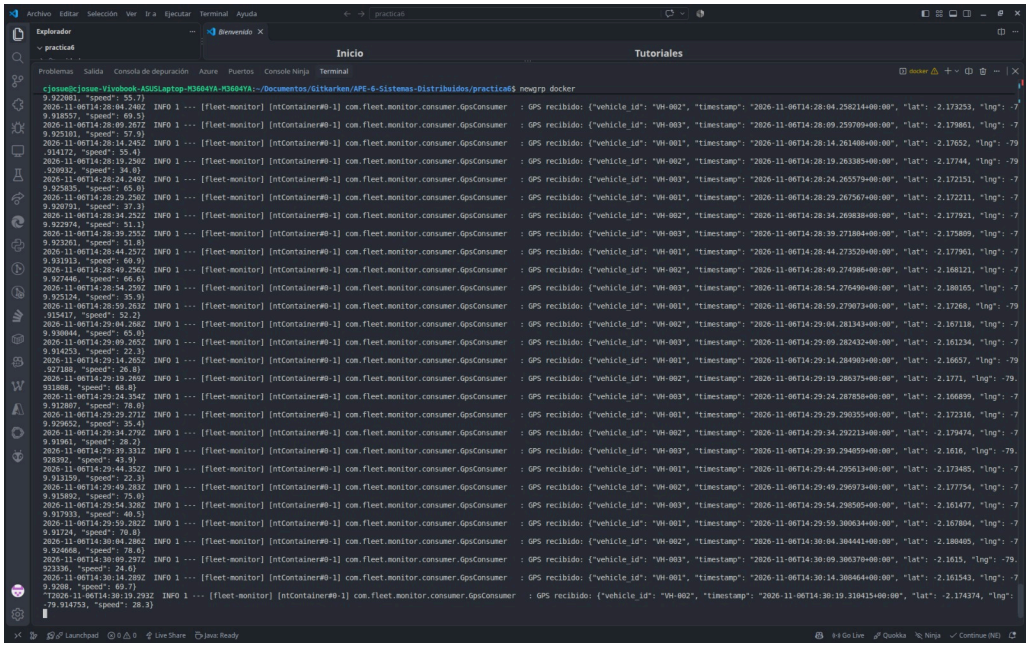


Figura 13. PC 2 (Carlos Granda): módulo GPS recibiendo la telemetría desde el servidor.

En la PC 2 se ejecutó el módulo *fleet-gps* conectado al broker RabbitMQ de la PC 1. En la captura se muestran los registros de *GpsConsumer* con el mensaje GPS recibido donde se ven los datos de ubicación (latitud, longitud y velocidad) de los tres vehículos lo que quiere decir que confirma que la telemetría llegó a la segunda computadora..

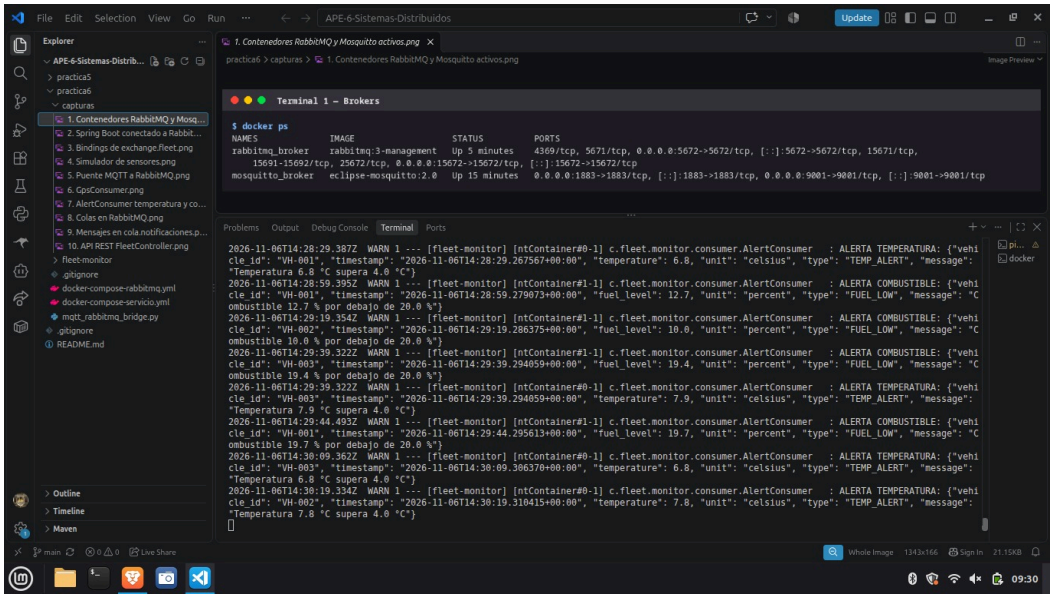


Figura 14. PC 3 (María Chuico): módulo de alertas procesando alertas de temperatura y combustible.

En la PC 3 se ejecutó el módulo *fleet-alertas*, en la imagen muestra los registros de *AlertConsumer* con los mensajes ALERTA TEMPERATURA y ALERTA COMBUSTIBLE que corresponden a las alertas leídas de *cola.alertas.temperatura* y *cola.combustible.nivel*a cada una de estas alertas se envían a *cola.notificaciones* para que la PC 4 la mostrara.

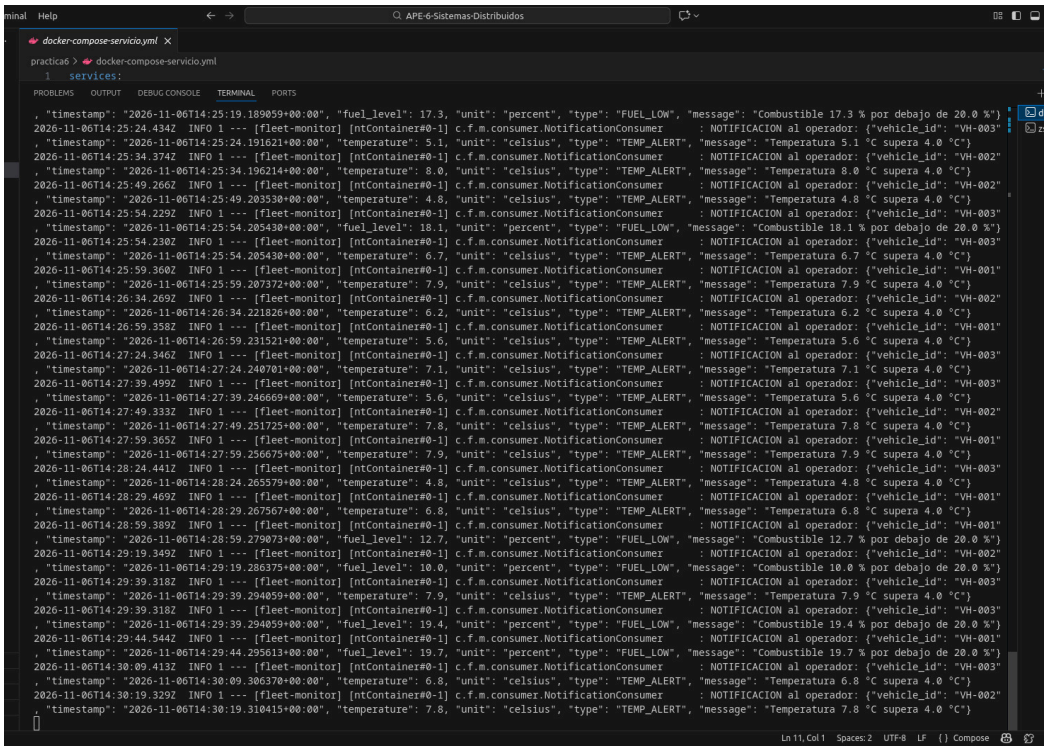


Figura 15. PC 4 (Juan Veintimilla): módulo de notificaciones mostrando las alertas al operador.

En la PC 4 se ejecutó el módulo *fleet-notificaciones*, en la captura se muestra los registros de *NotificationConsumer* con el mensaje NOTIFICACION al operador donde aparecen las alertas de temperatura (TEMP_ALERT) y de combustible (FUEL_LOW) con su vehículo, su valor y su mensaje lo que quiere decir que las alertas llegaron si llegaron a esta computadora.

6. Preguntas de Control

- **Explique los cuatro tipos de exchanges en RabbitMQ (direct, fanout, topic, headers) y proporcione un ejemplo de uso en el contexto de la flota logística?**

  Al trabajar con RabbitMQ se pudo comprender que existen diferentes maneras de distribuir los mensajes, aunque todas tienen el mismo propósito de facilitar la comunicación entre los servicios. El exchange direct permite enviar los mensajes a una cola específica mediante una routing key exacta, como sucede en nuestro proyecto cuando los datos GPS se dirigen a su respectiva cola. Por otro lado, fanout distribuye el mismo mensaje a todas las colas vinculadas, lo que podría servir para notificar simultáneamente a diferentes servicios sobre un evento de un vehículo. También está topic, que permite distribuir mensajes utilizando patrones en las routing keys, por ejemplo, para recibir información de todos los vehículos o solamente de uno en particular. Finalmente, headers realiza el enrutamiento utilizando las propiedades y encabezados de los mensajes, lo que permitiría clasificar alertas según su tipo o prioridad. Aunque los cuatro exchanges cumplen funciones diferentes, todos permiten organizar mejor la comunicación; en nuestra práctica utilizamos direct porque necesitábamos dirigir cada tipo de información hacia una cola determinada.

- **¿Qué es una cola durable en RabbitMQ y por qué es importante? ¿Qué sucedería si el broker se reinicia y las colas no son durables?**

  Una cola durable en RabbitMQ permite conservar su configuración incluso cuando el broker se apaga o se reinicia, por lo que resulta importante para mantener disponible la estructura de comunicación del sistema. Si las colas no fueran durables, al reiniciarse RabbitMQ desaparecerían y sería necesario crearlas nuevamente para continuar procesando los mensajes. Esto podría ocasionar interrupciones en la comunicación y afectar el funcionamiento de los servicios que dependen de ellas. Sin embargo, también comprendí que tener una cola durable no significa que todos sus mensajes se conservarán automáticamente, ya que estos deben configurarse como persistentes para permanecer almacenados ante un reinicio. Por eso, en un sistema de monitoreo de vehículos es importante considerar ambas configuraciones, especialmente cuando se trabaja con alertas relacionadas con la temperatura, el combustible o la ubicación.

- **Compare el patron pub/sub con el patron point-to-point. Cuales son las ventajas de pub/sub para el monitoreo de flotas?**

  Al comparar los patrones pub/sub y point-to-point se comprendió que ambos permiten intercambiar información, pero funcionan de una manera diferente. El patrón pub/sub permite que un publicador envíe un

mensaje y que varios suscriptores interesados puedan recibirlo, sin que el productor tenga que conocer directamente a cada uno. En cambio, el patrón point-to-point trabaja mediante colas donde los mensajes son distribuidos para que generalmente un solo consumidor procese cada mensaje, incluso cuando existen varios consumidores compitiendo por ellos. En el monitoreo de una flota vehicular, pub/sub resulta especialmente útil porque una misma actualización de GPS puede ser recibida por diferentes servicios, como uno encargado de mostrar la ubicación y otro de analizar los recorridos. Aunque ambos patrones tienen utilidad dependiendo de la necesidad, pub/sub ofrece mayor flexibilidad cuando se requiere compartir información entre varios componentes. En nuestra práctica se utiliza este enfoque mediante MQTT para transmitir los datos generados por los sensores.

- **¿Qué medidas de seguridad se pueden implementar en MQTT y RabbitMQ? Incluye autenticación, autorización, cifrado TLS y control de acceso.**

  Al implementar un sistema distribuido también es importante considerar la seguridad de la información, ya que los mensajes pueden contener datos importantes sobre la ubicación y las condiciones de los vehículos. Una de las medidas que se pueden aplicar es la autenticación mediante nombres de usuario y contraseñas, permitiendo comprobar quién intenta conectarse a los servicios. También se encuentra la autorización, que permite establecer qué acciones puede realizar cada usuario, evitando que todos tengan los mismos permisos para publicar o consumir mensajes. Por otra parte, el cifrado TLS permite proteger la información durante su transmisión, dificultando que otras personas puedan leerla si consiguen interceptar la comunicación. El control de acceso permite limitar los recursos disponibles para cada usuario, como las colas, los exchanges y los topics. Aunque estas medidas requieren una configuración adecuada, su implementación es importante para evitar accesos no autorizados, modificaciones de mensajes o generación de alertas falsas dentro del sistema.

7. Conclusiones

- Durante el desarrollo de la práctica, se implementó un sistema distribuido para el monitoreo de una flota vehicular, integrando MQTT, Mosquitto, RabbitMQ y Spring Boot. Esto permitió comprender cómo diferentes servicios pueden comunicarse e intercambiar información sin depender de una única aplicación.
- Se comprendió la importancia de los protocolos MQTT y AMQP dentro de la mensajería distribuida. Mediante MQTT se transmitieron los datos generados por los sensores de los vehículos, mientras que RabbitMQ permitió organizar y distribuir los mensajes mediante exchanges, colas y routing keys, lo que facilitó su procesamiento por parte de los diferentes consumidores.
- La implementación de los consumidores en Spring Boot permitió comprobar cómo se pueden procesar diferentes tipos de información de manera independiente. Se verificó la recepción de datos GPS y la generación de alertas relacionadas con la temperatura y el nivel de

combustible, demostrando la utilidad de los sistemas de mensajería para realizar el seguimiento de eventos en tiempo real.

- Finalmente, mediante las pruebas realizadas en RabbitMQ y los endpoints de la API REST, se comprobó la comunicación entre los componentes y la disponibilidad de la información de la flota. Esto permitió reforzar los conocimientos sobre integración de servicios, procesamiento de mensajes y desarrollo de aplicaciones distribuidas utilizando herramientas y tecnologías actuales.

8. Recomendaciones

- Se recomienda verificar el estado de los contenedores Docker y la conexión de los servicios RabbitMQ y Mosquitto antes de iniciar las aplicaciones, ya que esto permite evitar problemas de comunicación durante la ejecución de la práctica.
- Es importante mantener una correcta organización de los exchanges, las colas y las routing keys en RabbitMQ, utilizando nombres claros para cada tipo de mensaje. Esto facilita la identificación de los datos y permite realizar cambios en el sistema sin generar confusiones en su distribución.
- Se recomienda realizar pruebas con diferentes valores de temperatura y niveles de combustible para comprobar el comportamiento de las alertas ante distintas situaciones. También es conveniente revisar los registros de ejecución para identificar posibles errores en la recepción y el procesamiento de los mensajes.
- Para futuras implementaciones, se recomienda incorporar medidas de seguridad en la comunicación entre los servicios, como autenticación de usuarios, control de permisos y cifrado mediante TLS en MQTT y RabbitMQ. Esto ayudaría a proteger la información transmitida y evitar accesos no autorizados al sistema.

9. Declaración del uso de la IA

Para el desarrollo de la presente práctica se utilizaron las herramientas de inteligencia artificial ChatGPT y Claude como apoyo durante el proceso de trabajo. Estas herramientas se emplearon para consultar información, aclarar dudas técnicas, orientar la preparación y configuración del entorno de desarrollo, así como comprender el funcionamiento de las tecnologías utilizadas, como RabbitMQ, MQTT y Spring Boot. También se utilizaron como apoyo para organizar las ideas y mejorar la redacción del informe. Sin embargo, la implementación, ejecución, configuración de los servicios y realización de las pruebas fueron desarrolladas por los integrantes del grupo, quienes revisaron y verificaron la información utilizada.

10. Bibliografía

Semana 5. Guaman, Jose. 2026

11. Anexos

- **Link del repositorio:**
  https://github.com/Michu117/APE-6-Sistemas-Distribuidos
- **Enlaces VIdeos:**
  A. **María Chuico**
     https://youtu.be/AjIIbLktyEg
  B. **Carlos Granda**
     https://youtu.be/Sa2Ght43lll
  C. **Fabricio Ruiz**
     https://youtu.be/a_-AQZLY45g
  D. **Juan Veintimilla**
     https://youtu.be/M5HxXiuAGWU

- **Fotos Laboratorio**



Figura 16. Integrantes del grupo durante la práctica en el laboratorio.

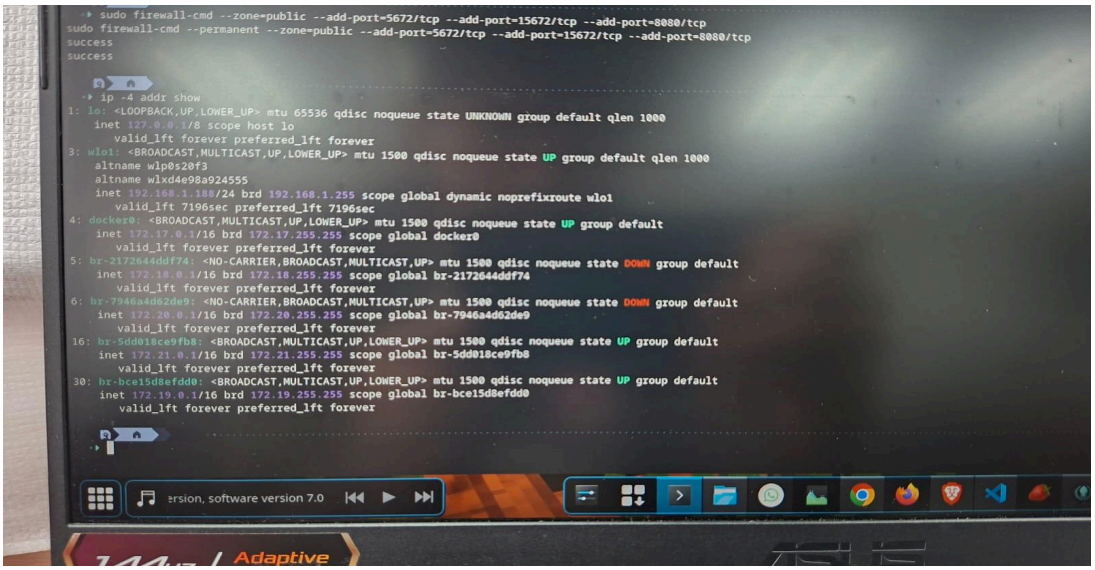


Figura 17. Configuración del firewall y direcciones IP de la PC servidor.

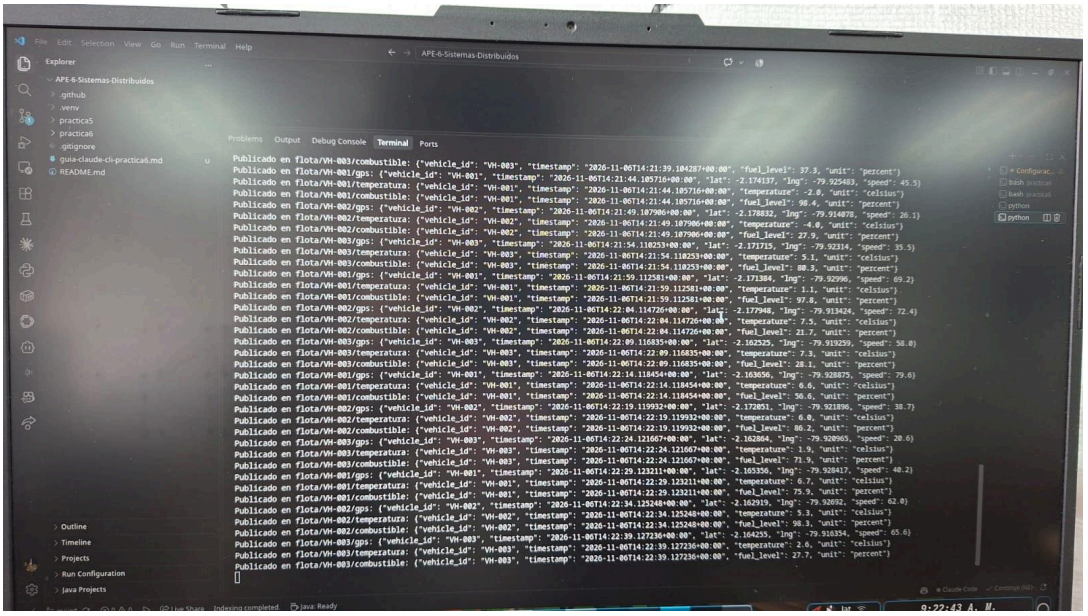


Figura 18. PC 1 (Fabricio Ruiz): simulador de sensores publicando telemetría por MQTT.

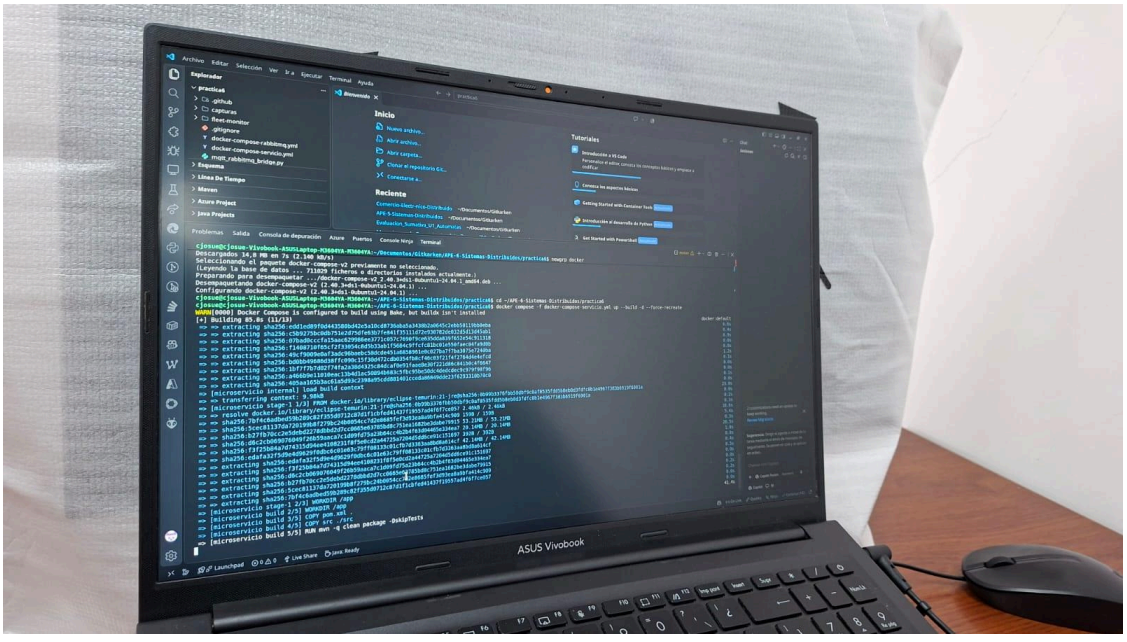


Figura 19. PC 2 (Carlos Granda): construcción del contenedor del módulo fleet-gps con Docker Compose.

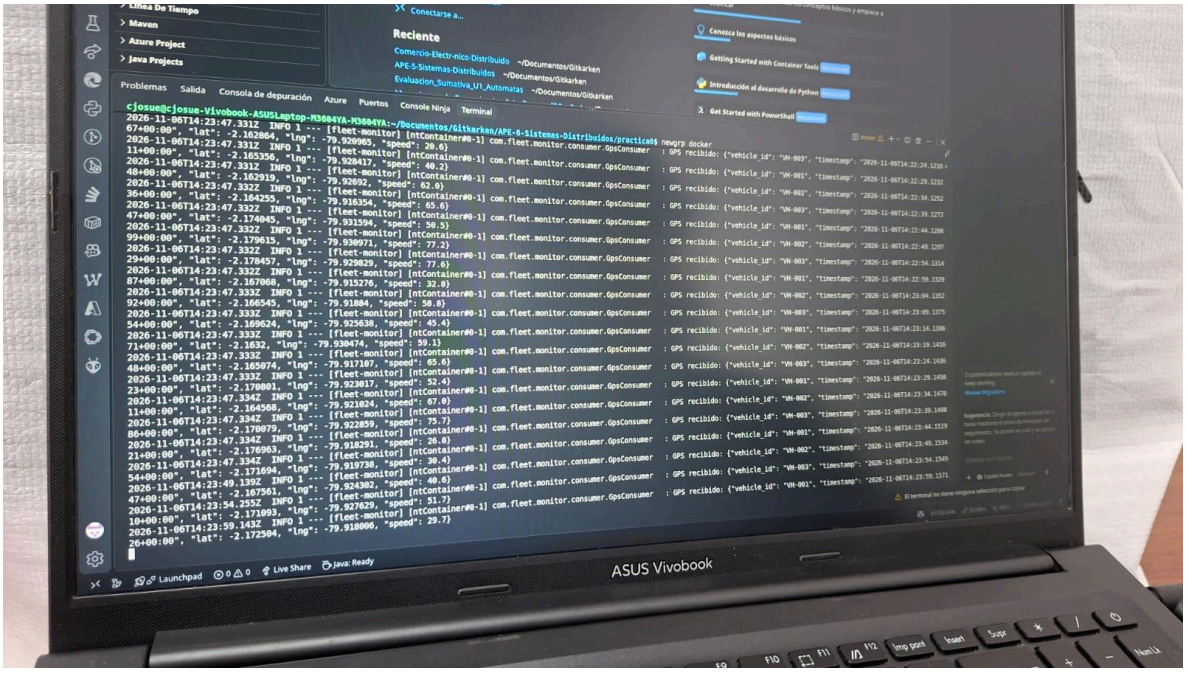


Figura 20. PC 2 (Carlos Granda): GpsConsumer recibiendo los mensajes de ubicación de los vehículos.

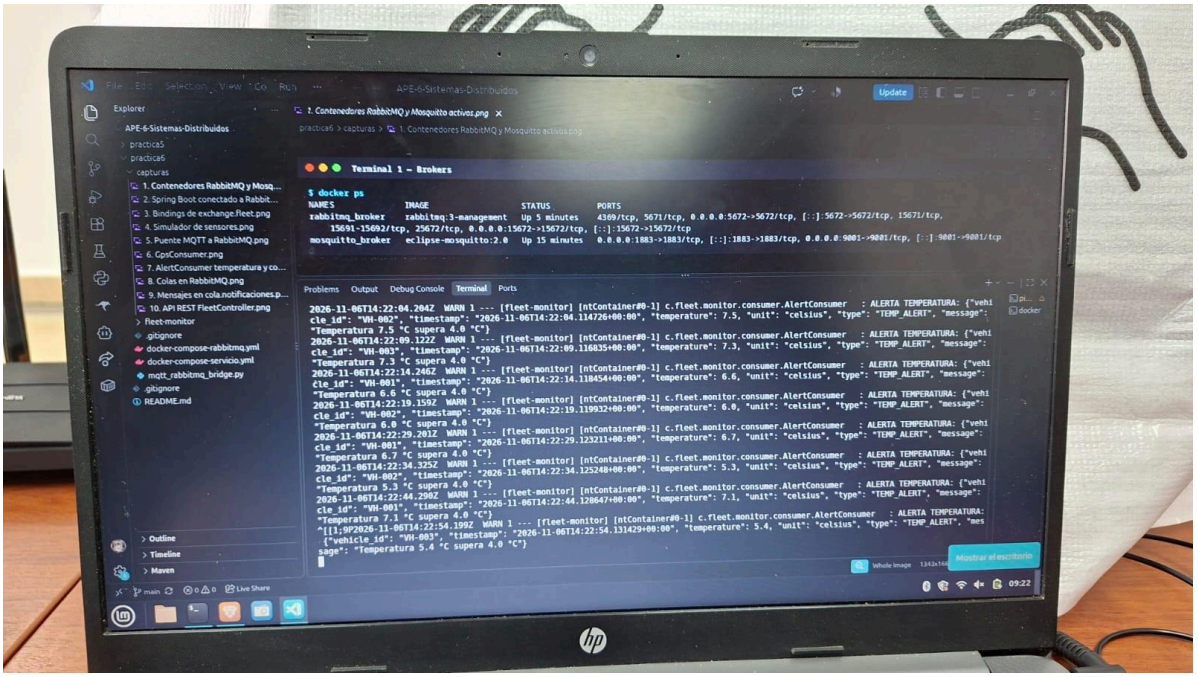


Figura 21. PC 3 (María Chuico): módulo de alertas procesando alertas de temperatura.

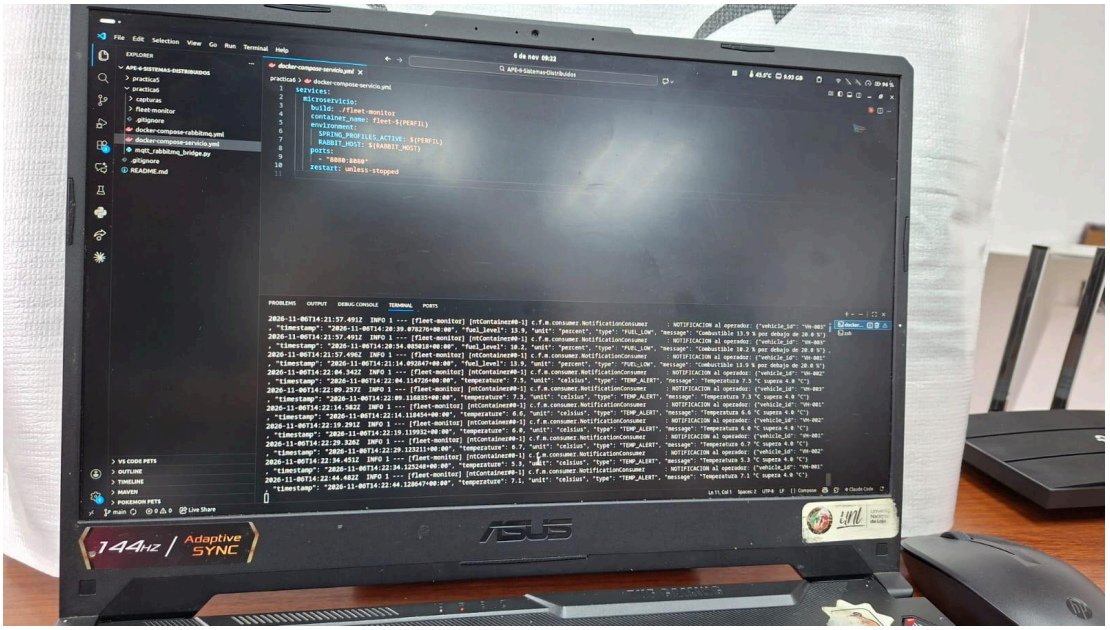


Figura 22. PC 4 (Juan Veintimilla): módulo de notificaciones mostrando las alertas al operador.



Figura 23. Router TP-Link utilizado para la red local del despliegue.

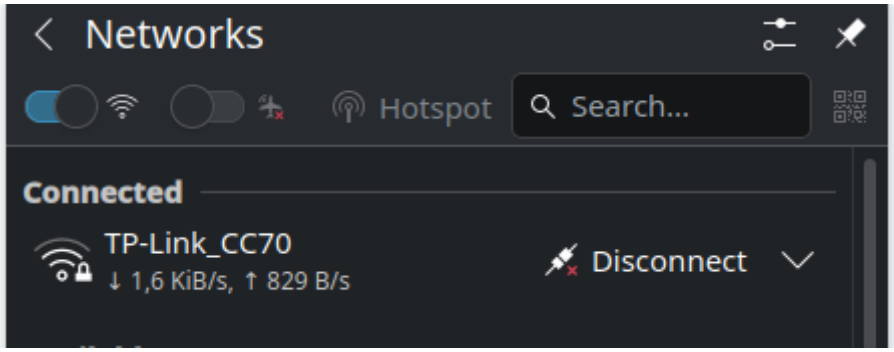


Figura 24. Conexión de las computadoras a la red TP-Link_CC70.